Name:Siheng Zhao

Project final write-up about two/three body simulation program

Introduction of the program:

My program is about simulating two/three body problems for planet(massive object), as we all know that, we can't find a analytical solution for three/multi body problem, but we can use numerical integration to find a special case solution or to simulate the motion of the system roughly.(the complete code was in the last page)

Program framework and general idea:

First of all, in the beginning of the code, I set up some global variables, they are gravitational constant G=6.67430e-11, time step dt=86400s(one day), and simulation time t=3650(ten years) respectively(Figure 1).

```
//the gravitational constant G
double G = 6.67430e-11;

//set the update time interval
double dt = 86400; //86400s = 1 day

//set how long do you want to simulate
int t = 3650; //in days

//set the parameter for what time is now
int day = 0;
```

Figure 1

Then, I create a "struct" named "star", and I put the basic parameters(mass m, radius r, xy position&velocity) of the star/body in to the "struct" so it can make me call all the parameters more easier when I was using a function(Figure 2**).**

```
//set the parameter for the star
struct star{
    double m;
    double r;
    float x , y;
    float vx , vy;
};

star s1, s2, s3;
```

Figure 2

I also set up some function before doing the calculation, they are:

1. **"double d(const star &a, const star &b)" for calculating the distance between star a&b((Figure 3 red box));**

2. **"double v(const star &a)" for calculating the velocity for star a(Figure 3 yellow box);**

3. **"void model_selection()" for select how many bodies dose the user want to simulate(2 or 3), and which model dose the user want to pick(The default demo model that comes with the program or the user self-select model (Figure5)), and user can select the model by the "switch" function; below model_selection is the acceleration computing function energy computing function and momentum computing function;**

4. **"void report2/3(const star &a, const star &b, const star &c)"for reporting the state of the star(position, velocity and it's component, acceleration and it's component(Figure 4 red box), kinetic energy Ek, potential energy Eu, total energy E(Figure 4 yellow box), and total momentum P(Figure 4 green box)).**

```
//distance between two star
double d(const star &a, const star &b){
    double distance;
    distance = sqrt(pow(a.x-b.x,2)+pow(a.y-b.y,2));
    return distance;
}
```
distance

```
//speed of a star
double v(const star &a){
    return sqrt(pow(a.vx,2)+pow(a.vy,2));
}
```
speed

Figure 3

```
[zhaos111@phys-ugrad proj1]$ orbit
2 for two body problem simulation
3 for three body problem simulation

3
three body problem model selection:

1 for user-defined model
2 for the third Lagrangian Points with a star at rest
3 for the third Lagrangian Points with a moving star
4 for earth-sun-mars system
5 for resonance system(simulate for 300 earth-year, about 1-2min in real world)
6 for a star passing a binary system
7 for a planet orbiting a binary system

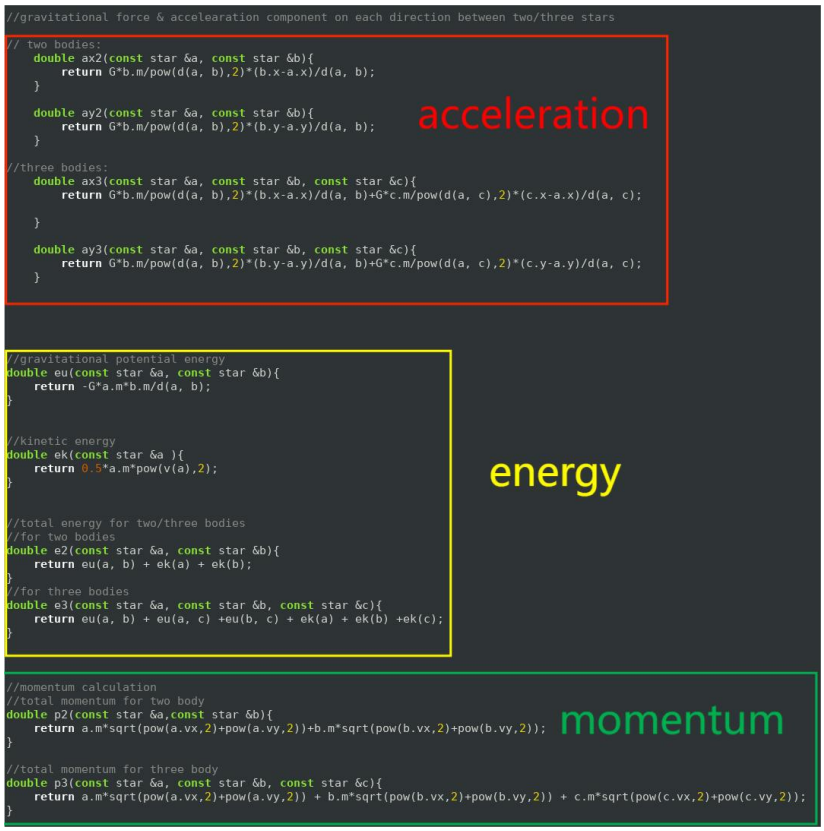
special case solution for three body problems:
8 for stable three body problem's special case solution code name-'Figure 8'
9 for stable three body problem's special case solution code name-'Free Fall F1'
10 for stable three body problem's special case solution code name-'Double Ovals'

unstable system:
100 for unstable system 1
101 for unstable system 2
102 for unstable system 3
103 for unstable system 4
```
body_num selection

model selection

Figure 5

Name:Siheng Zhao




```
//gravitational force & accelearation component on each direction between two/three stars
// two bodies:
    double ax2(const star &a, const star &b){
        return G*b.m/pow(d(a, b),2)*(b.x-a.x)/d(a, b);
    }

    double ay2(const star &a, const star &b){
        return G*b.m/pow(d(a, b),2)*(b.y-a.y)/d(a, b);
    }
//three bodies:
    double ax3(const star &a, const star &b, const star &c){
        return G*b.m/pow(d(a, b),2)*(b.x-a.x)/d(a, b)+G*c.m/pow(d(a, c),2)*(c.x-a.x)/d(a, c);

    }

    double ay3(const star &a, const star &b, const star &c){
        return G*b.m/pow(d(a, b),2)*(b.y-a.y)/d(a, b)+G*c.m/pow(d(a, c),2)*(c.y-a.y)/d(a, c);
    }
```

```
//gravitational potential energy
double eu(const star &a, const star &b){
    return -G*a.m*b.m/d(a, b);
}


//kinetic energy
double ek(const star &a ){
    return 0.5*a.m*pow(v(a),2);
}


//total energy for two/three bodies
//for two bodies
double e2(const star &a, const star &b){
    return eu(a, b) + ek(a) + ek(b);
}
//for three bodies
double e3(const star &a, const star &b, const star &c){
    return eu(a, b) + eu(a, c) +eu(b, c) + ek(a) + ek(b) +ek(c);
}
```

```
//momentum calculation
//total momentum for two body
double p2(const star &a,const star &b){
    return a.m*sqrt(pow(a.vx,2)+pow(a.vy,2))+b.m*sqrt(pow(b.vx,2)+pow(b.vy,2));
}

//total momentum for three body
double p3(const star &a, const star &b, const star &c){
    return a.m*sqrt(pow(a.vx,2)+pow(a.vy,2)) + b.m*sqrt(pow(b.vx,2)+pow(b.vy,2)) + c.m*sqrt(pow(c.vx,2)+pow(c.vy,2));
}
```

Figure 4

ODE explanations:

The numerical integration method I used was leapfrog, the reason I choose this method is because, it can better maintenance the conservation of energy and momentum.

First, set up the variables and arrays that we need later, and initialized the first terms in the arrays.(Figure 7)

Second, use a for loop to start calculation, and use 1/2 dt as the time step for the first update of the speed(Figure 6 yellow box), V(t+0.5t)=V(t)+a(x(t))*0.5dt. And then use the V(t+0.5t) that we get from the first update to compute the new whole step position(Figure 6 green box), x(t+1dt)=x(t)+1dt*V(t+0.5dt). by getting the new position update, we are be able to know whether the two star collided with each other(Figure 6 pink box), colliding test can avoid floating point exception(if distance between a&b <= ra+rb, then break the program). Now, by knowing the new position, we can finish the last half-step for speed updating(Figure 6 blue box), V(t+1dt)=V(t+0.5dt)+0.5dt*a(x(t+1dt)) .

① **Initialize v=star.v**

② **Initialize x,y,z=star.x,y,z**

③ **Set condition for "for loop": i=1; i<=t; i++.**

④ **First half-step update: V(t+0.5t)=V(t)+a(x,y,z(t))*0.5dt**

⑤ **Whole step update the position: x,y,z(t+1dt)=x,y,z(t)+1dt*V(t+0.5dt)**

⑥ **Check whether any star collide with other.**

If (d(star a, star b) <= ra+rb) :break

Else:continue

⑦ **Update the last half-step for speed with the new position and acceleration:**

V(t+1dt)=V(t+0.5dt)+0.5dt*a(x,y,z(t+1dt))

⑧ **Report the state of the stars, and store the energy momentum, into the arrays.**

⑨ **Repeat step 4, until reach the "for loop" condition.**

V(t+0.5t): the first half-step of v

x,y,z(t+1dt): the whole step of position

V(t+1dt): the final whole step of v

Name:Siheng Zhao

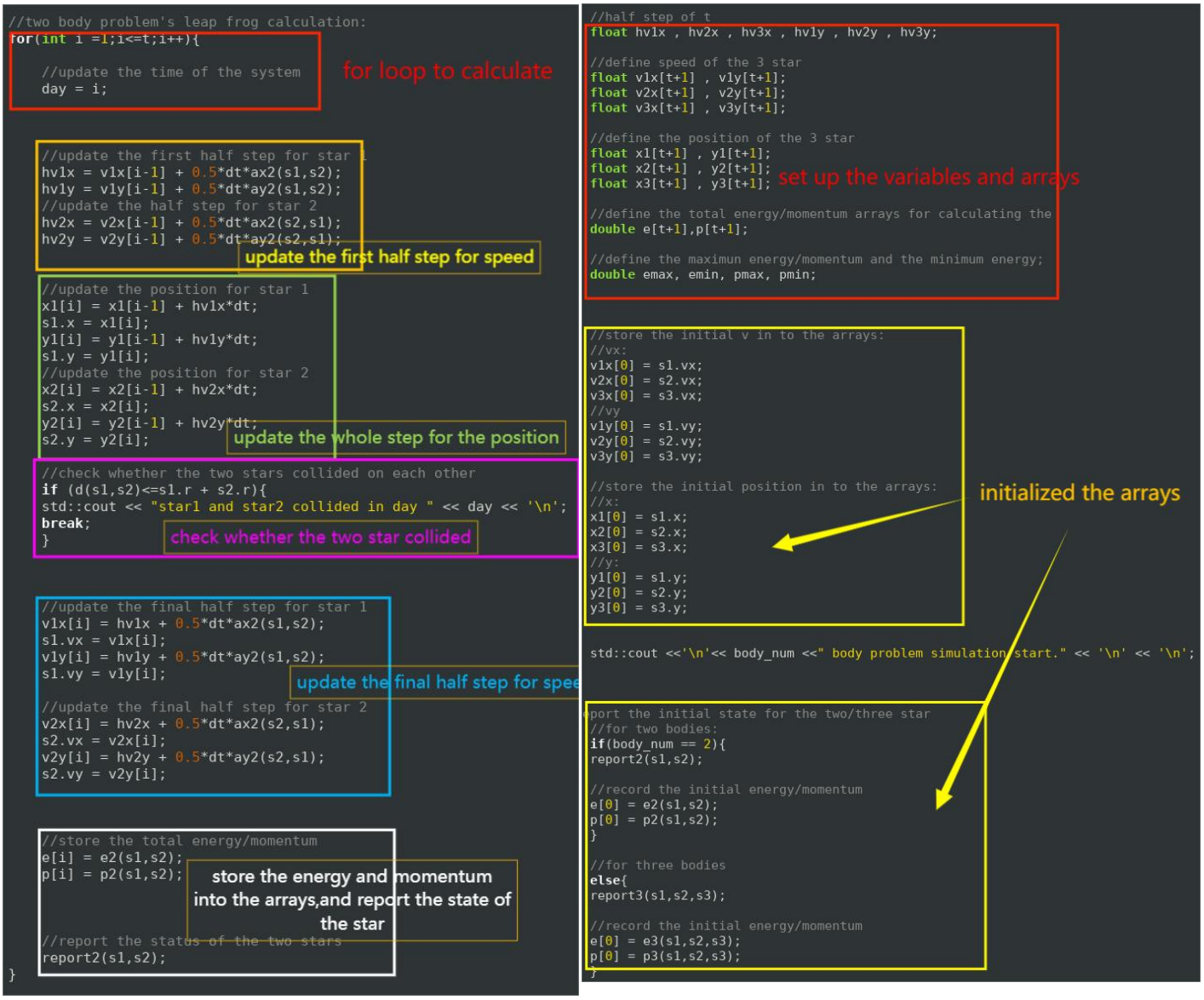


Figure 7

Name:Siheng Zhao

Code testing:

There are two way to test whether my code is running well:

1. **The conservation of energy&momentum. In some simple orbit(like the earth-sun system, or the binary star system), the energy&momentum should almost conserved(due to the numerical integration method, there will still be some error during the simulation), we can check how large the total energy&momentum changed to tell whether it conserved or not, so I put a function in the end of the program to collect the energy&momentum data in each step, and find out Emax&Pmax and Emin&Pmin, use (Emax-Emin)/E0 or(Pmax-Pmin)/P0 to tell the error.(Figure 8, Figure 9)**

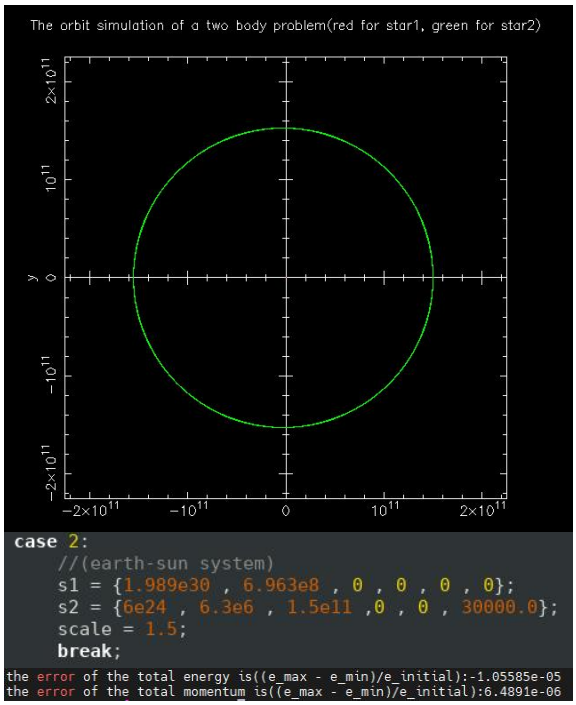


Figure 8

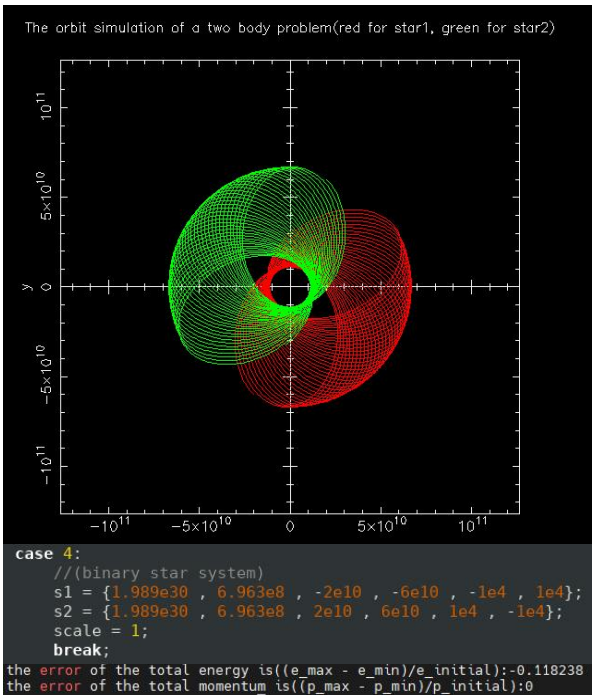


Figure 9

2. **See whether the orbit is stable, and compare it with the real life, in this picture(Figure 10), earth orbiting the sun and sun has a small speed pointing to x positive, radius 1AU(earth to sun distance), so if the model is correct, simulating 10 years earth will goes around the sun ten times.**

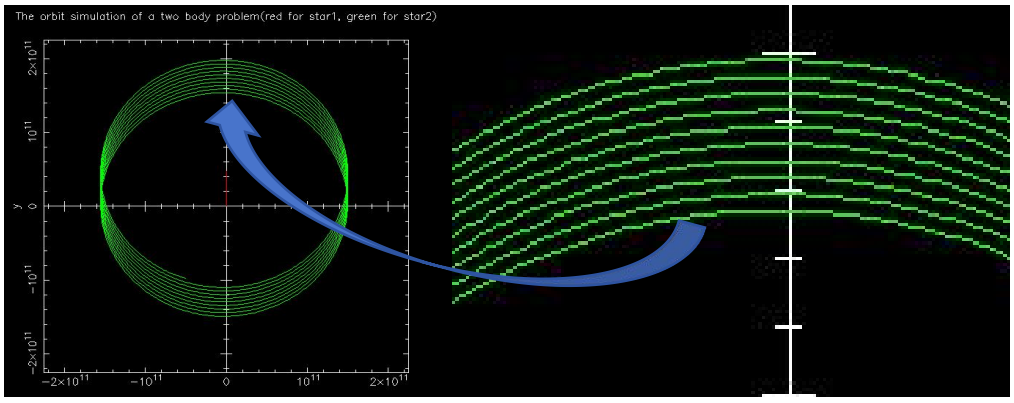


Figure 10

3. **But for some unstable system, or complex system, the conservation of energy and momentum will not acting pretty well(Figure11/12 red box). but we can still solve it by decreasing the time step. In the next time, I decrease the time step from 86400s to 8640s and increase the total simulation step from 3650 to 36500, so can make sure we still simulate the same time. And the result is, the floating of the energy and momentum drop sharply(Figure 11/12 blue box)**

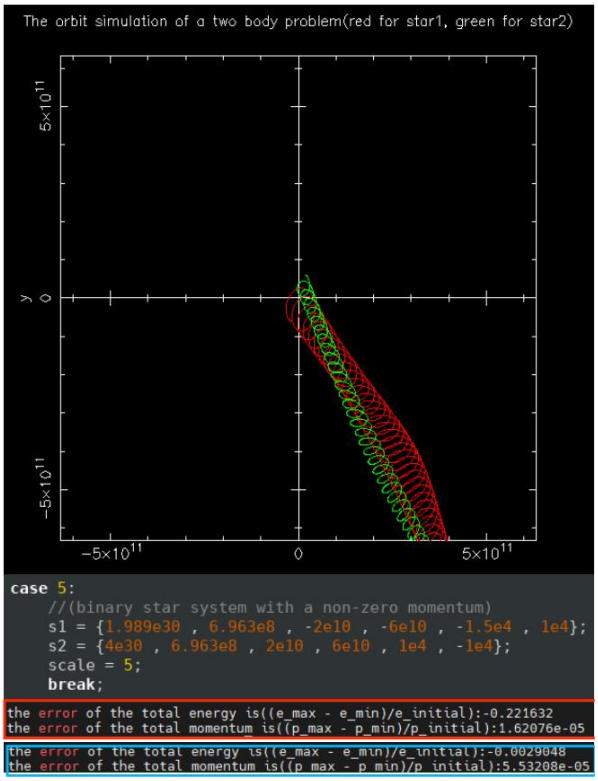


Figure 11

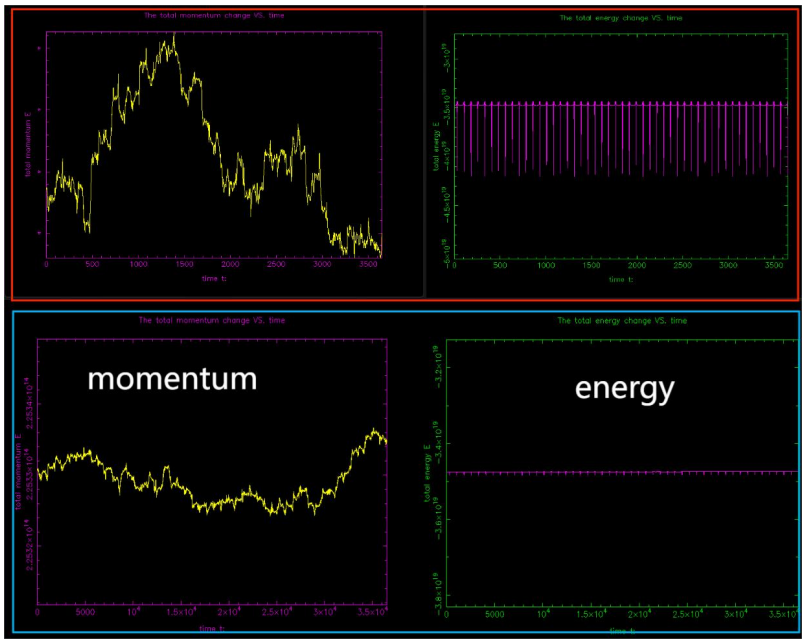


Figure 12

Code extension:

I expand my code from simulating two body problem to three body problems. User can select which one do they want to simulate in the beginning of the program.

Also I make some adjustment on the function to make them become more suitable for three body simulation(Figure 13, red for two body blue for three body)

```
//gravitational force & accelearation component on each direction between two/three stars
// two bodies:
    double ax2(const star &a, const star &b){
        return G*b.m/pow(d(a, b),2)*(b.x-a.x)/d(a, b);
    }

    double ay2(const star &a, const star &b){
        return G*b.m/pow(d(a, b),2)*(b.y-a.y)/d(a, b);
    }
//three bodies:
    double ax3(const star &a, const star &b, const star &c){
        return G*b.m/pow(d(a, b),2)*(b.x-a.x)/d(a, b)+G*c.m/pow(d(a, c),2)*(c.x-a.x)/d(a, c);

    }

    double ay3(const star &a, const star &b, const star &c){
        return G*b.m/pow(d(a, b),2)*(b.y-a.y)/d(a, b)+G*c.m/pow(d(a, c),2)*(c.y-a.y)/d(a, c);
    }



//gravitational potential energy
double eu(const star &a, const star &b){
    return -G*a.m*b.m/d(a, b);
}


//kinetic energy
double ek(const star &a ){
    return 0.5*a.m*pow(v(a),2);
}

//total energy for two/three bodies
//for two bodies
double e2(const star &a, const star &b){
    return eu(a, b) + ek(a) + ek(b);
}
//for three bodies
double e3(const star &a, const star &b, const star &c){
    return eu(a, b) + eu(a, c) +eu(b, c) + ek(a) + ek(b) +ek(c);
}

//momentum calculation
//total momentum for two body
double p2(const star &a,const star &b){
    return a.m*sqrt(pow(a.vx,2)+pow(a.vy,2))+b.m*sqrt(pow(b.vx,2)+pow(b.vy,2));
}
//total momentum for three body
double p3(const star &a, const star &b, const star &c){
    return a.m*sqrt(pow(a.vx,2)+pow(a.vy,2)) + b.m*sqrt(pow(b.vx,2)+pow(b.vy,2)) + c.m*sqrt(pow(c.vx,2)+pow(c.vy,2));
}
```

Figure 13

```
//update the first half step for star 1
hv1x = v1x[i-1] + 0.5*dt*ax3(s1,s2,s3);
hv1y = v1y[i-1] + 0.5*dt*ay3(s1,s2,s3);
//update the first half step for star 2
hv2x = v2x[i-1] + 0.5*dt*ax3(s2,s1,s3);
hv2y = v2y[i-1] + 0.5*dt*ay3(s2,s1,s3);
//update the first half step for star 3
hv3x = v3x[i-1] + 0.5*dt*ax3(s3,s1,s2);
hv3y = v3y[i-1] + 0.5*dt*ay3(s3,s1,s2);


//update the position for star 1
x1[i] = x1[i-1] + hv1x*dt;
s1.x = x1[i];
y1[i] = y1[i-1] + hv1y*dt;
s1.y = y1[i];
//update the position for star 2
x2[i] = x2[i-1] + hv2x*dt;
s2.x = x2[i];
y2[i] = y2[i-1] + hv2y*dt;
s2.y = y2[i];
//update the position for star 3
x3[i] = x3[i-1] + hv3x*dt;
s3.x = x3[i];
y3[i] = y3[i-1] + hv3y*dt;
s3.y = y3[i];

//check whether any of the the two stars collided on each other
if (d(s1,s2)<=s1.r + s2.r){
std::cout << "star1 and star2 collided in day " << day << '\n';
break;
}
else if(d(s1,s3)<=s1.r + s3.r){
std::cout << "star1 and star3 collided in day " << day << '\n';
break;
}
else if(d(s3,s2)<=s3.r + s2.r){
std::cout << "star3 and star2 collided in day " << day << '\n';
break;
}

//update the final half step for star 1
v1x[i] = hv1x + 0.5*dt*ax3(s1,s2,s3);
s1.vx = v1x[i];
v1y[i] = hv1y + 0.5*dt*ay3(s1,s2,s3);
s1.vy = v1y[i];

//update the final half step for star 2
v2x[i] = hv2x + 0.5*dt*ax3(s2,s1,s3);
s2.vx = v2x[i];
v2y[i] = hv2y + 0.5*dt*ay3(s2,s1,s3);
s2.vy = v2y[i];

//update the final half step for star 3
v3x[i] = hv3x + 0.5*dt*ax3(s3,s1,s2);
s3.vx = v3x[i];
v3y[i] = hv3y + 0.5*dt*ay3(s3,s1,s2);
s3.vy = v3y[i];
```

Figure 14

I still take leapfrog as the numerical integral method(Figure 14, red box for the terms added)

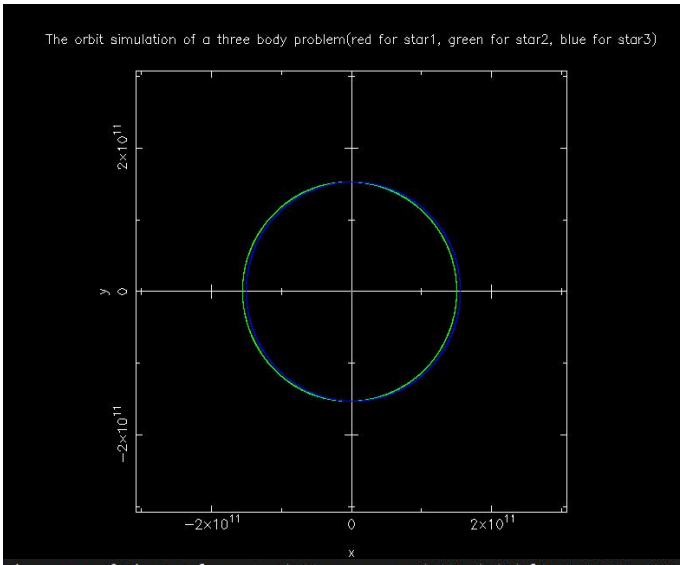
extension code exploration & demo:

The following were some good value that a recorded during the test:

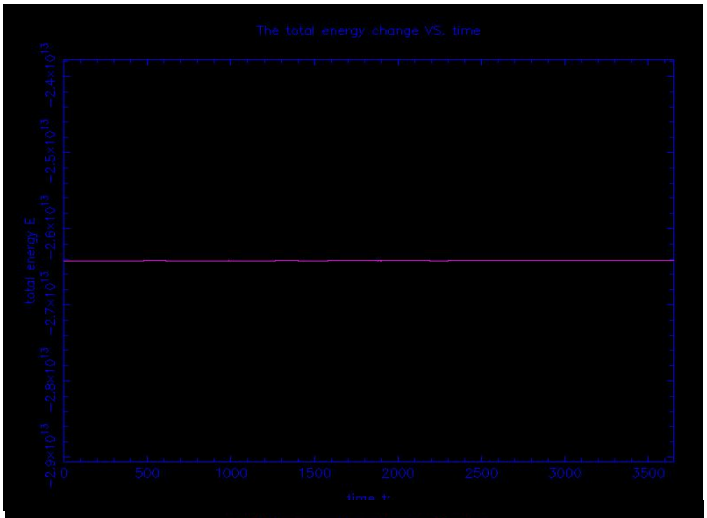
We can divide them into two sort, one is periodic, and another is non-periodic:

periodic:

The Third Lagrange point(Figure 15):




```
the error of the total energy is((e_max - e_min)/e_initial):-1.15891e-05
the error of the total momentum is((p_max - p_min)/p_initial):5.32739e-06
case 2:
    //(earth-sun system)
    s1 = {1.989e30 , 6.963e8 , 0 , 0 , 0 , 0};
    s2 = {6e24 , 6.3e6 , 1.5e11 ,0 , 0 , 30000.0};
    scale = 1.5;
    break;
```



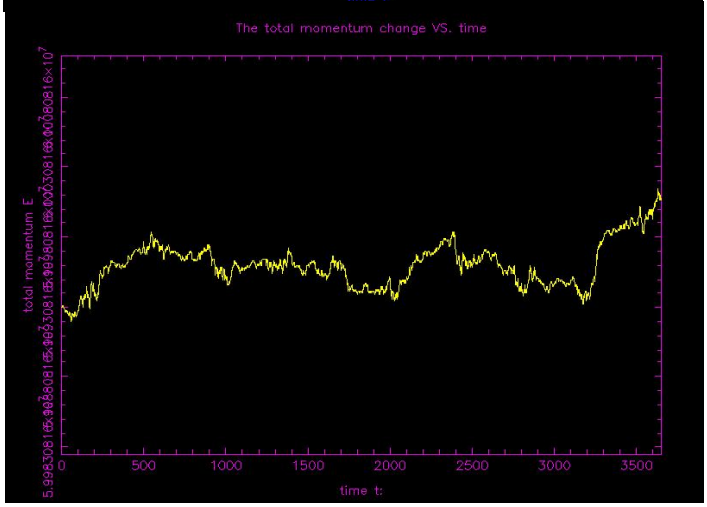
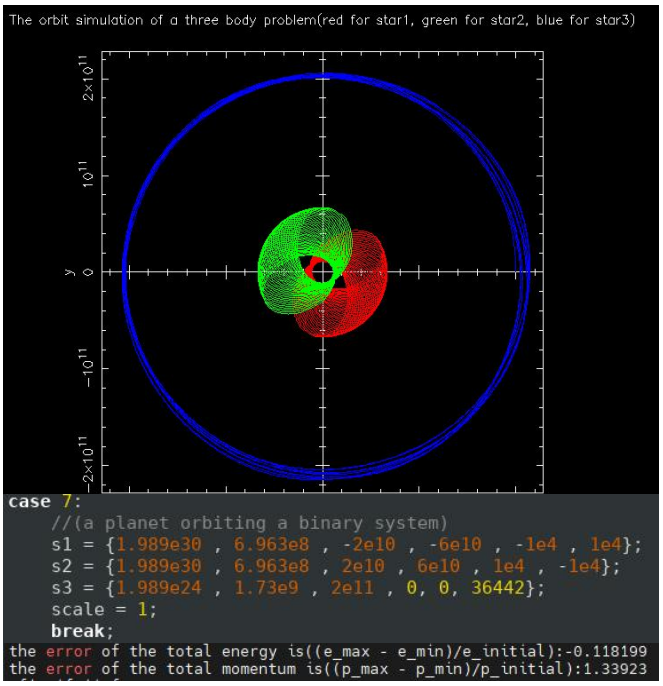


Figure 15

Name:Siheng Zhao

A planet orbiting around a binary star system(Figure 16):



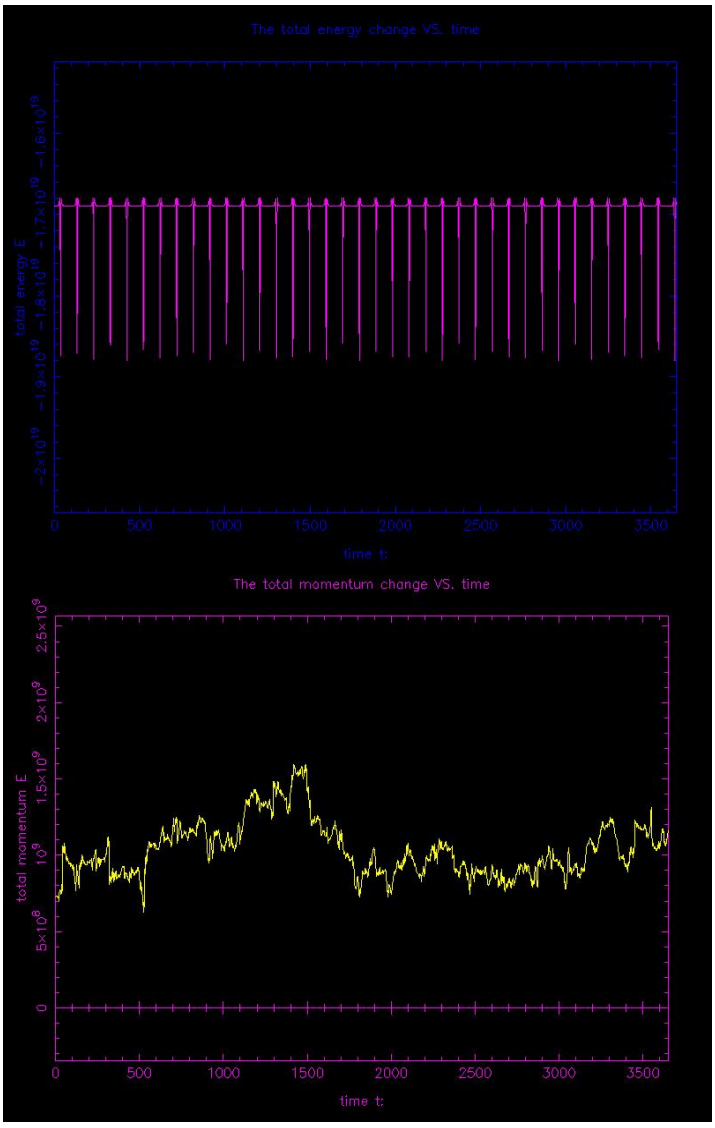
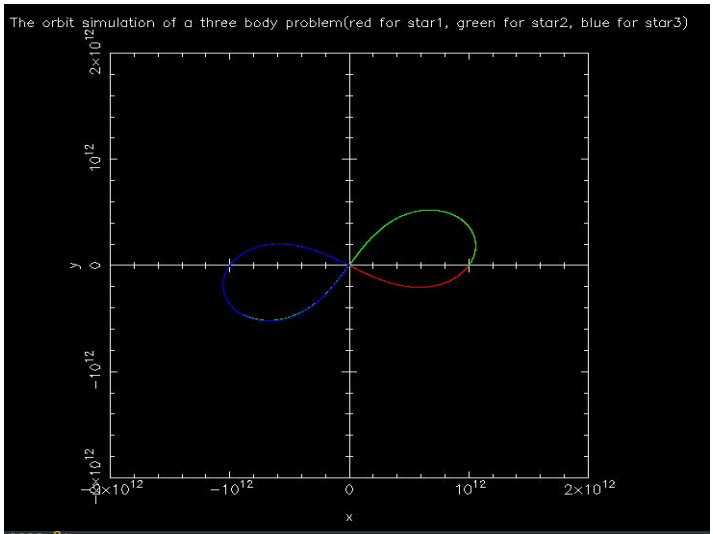


Figure 16

Name:Siheng Zhao

Three body problem special case solution-‘FIGURE-8’ (Figure 17)：




```
case 8:
    ///(stable orbit-'Figure "8"')
    s1 = {1*1.5e30 , 6.963e8 , -1e12 , 0 , 0.3471168881e4 , 0.5327249454e4};
    s2 = {1*1.5e30 , 6.963e8 , 1e12 , 0 , 0.3471168881e4 , 0.5327249454e4};
    s3 = {1*1.5e30 , 6.963e8 , 0 , 0 , -0.3471168881*2e4 , -0.5327249454*2e4};
    scale = 1;
    break;
the error of the total energy is((e_max - e_min)/e_initial):-6.25043e-06
the error of the total momentum is((p_max - p_min)/p_initial):0
```

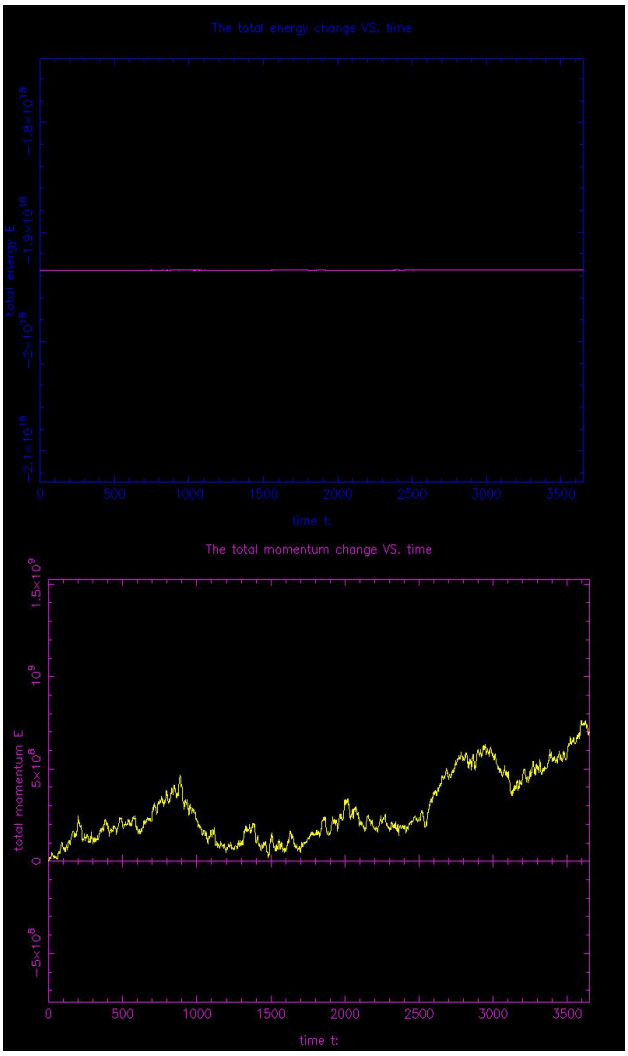
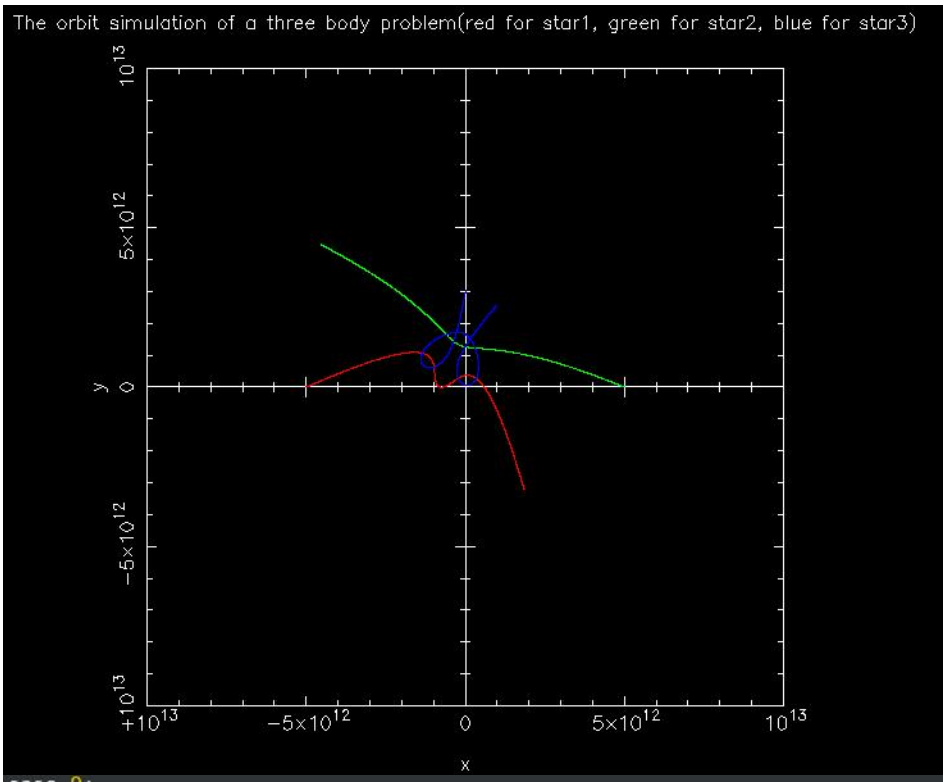


Figure 17

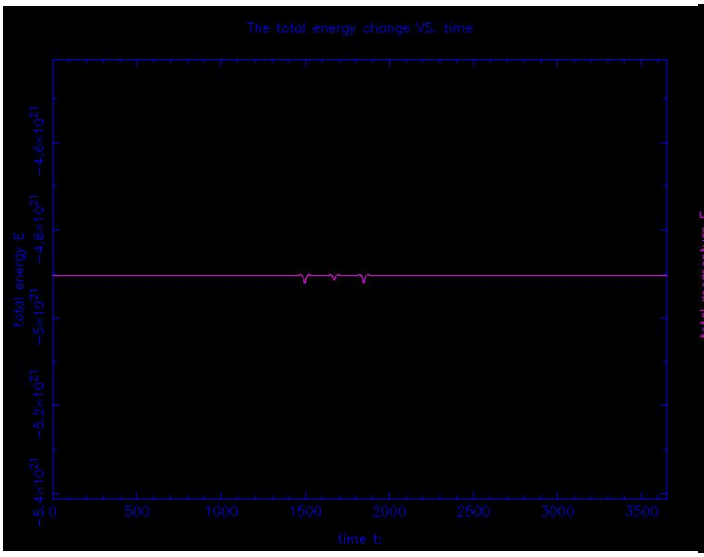
Three body problem special case solution-'Free Fall' (Figure 18):

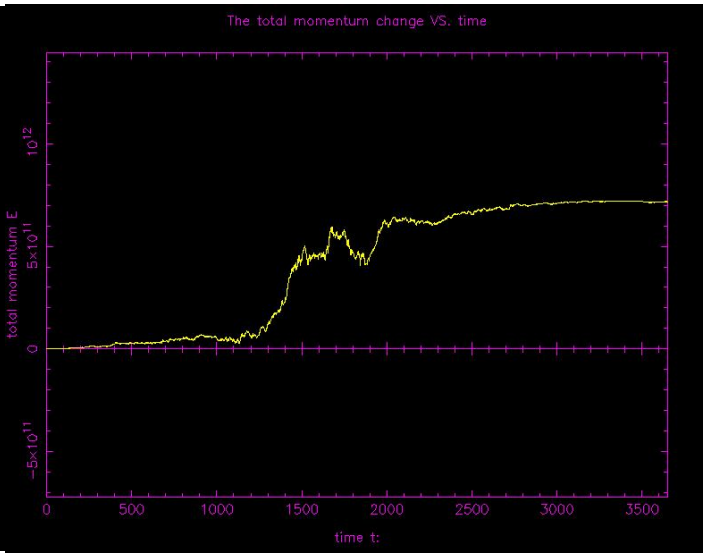



```
case 9:
    //(stable orbit-'Free Fall F1')
    s1 = {1*1.5e32 , 6.963 , -0.5e13 , 0 , 0 , 0};
    s2 = {0.8*1.5e32 , 6.963 , 0.5e13 , 0 , 0 , 0};
    s3 = {0.8*1.5e32 , 6.963 , 0.0009114239e13 , 0.3019805958e13 , 0 , 0};
    scale = 1;
    break;
```

```
the error of the total energy is((e_max - e_min)/e_initial):-0.00380543
the error of the total momentum is((p max - p min)/p initial):0
```

Figure 18





Name:Siheng Zhao

Three body problem special case solutlsion-‘Free Fall’ (Figure 19)：

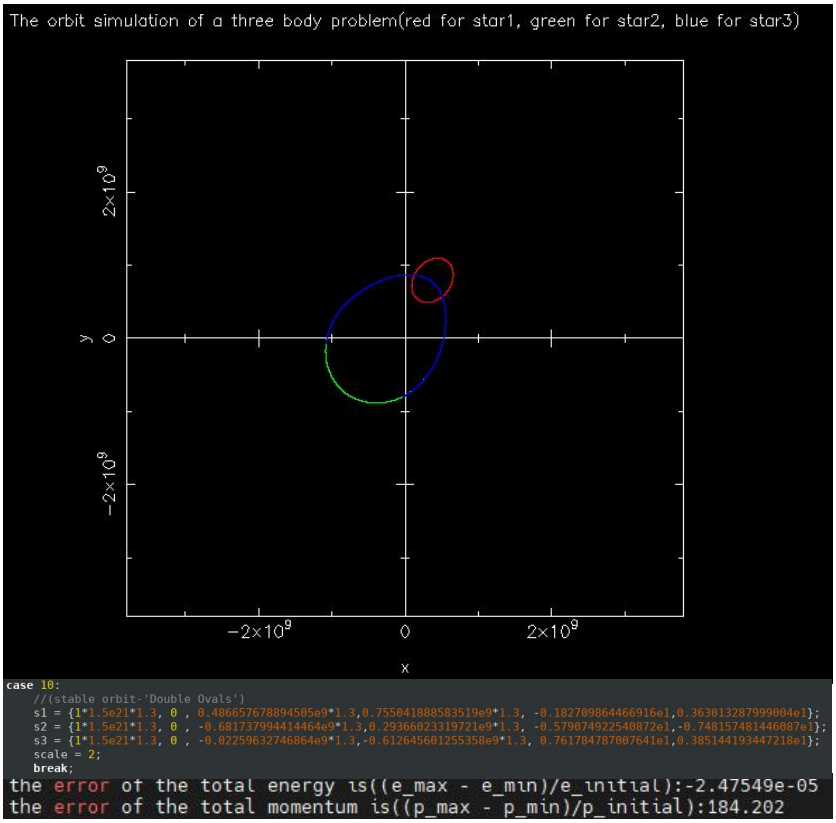
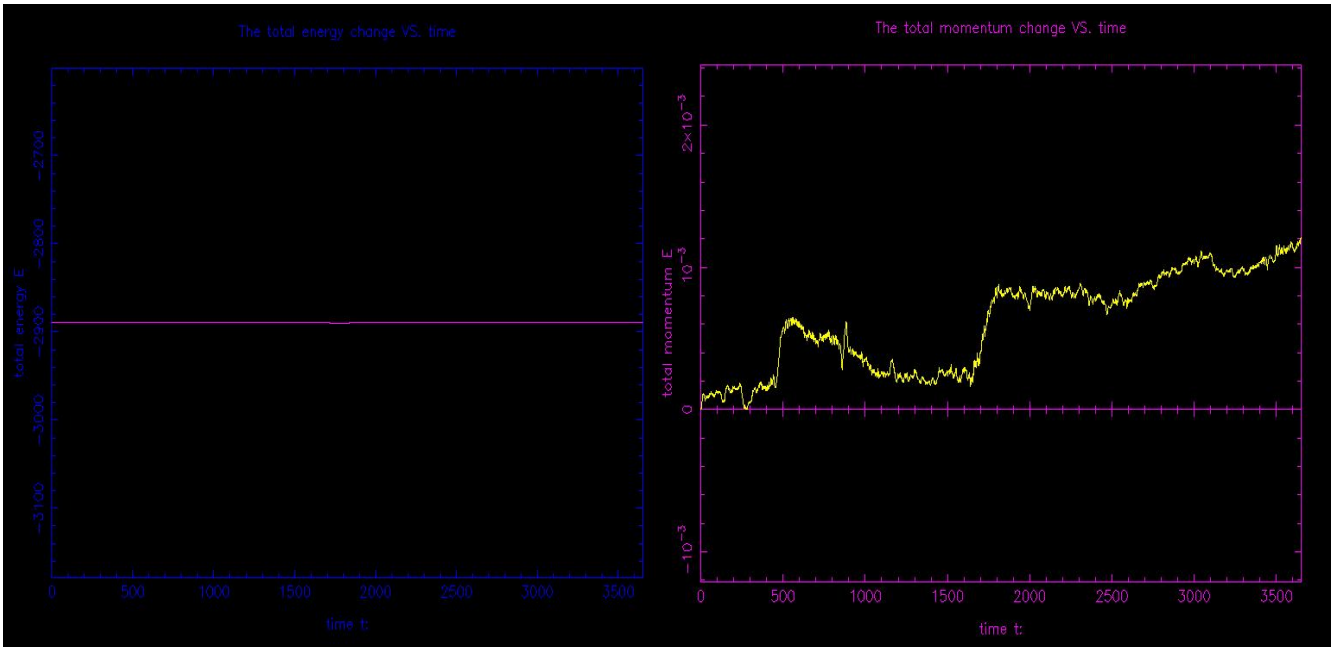


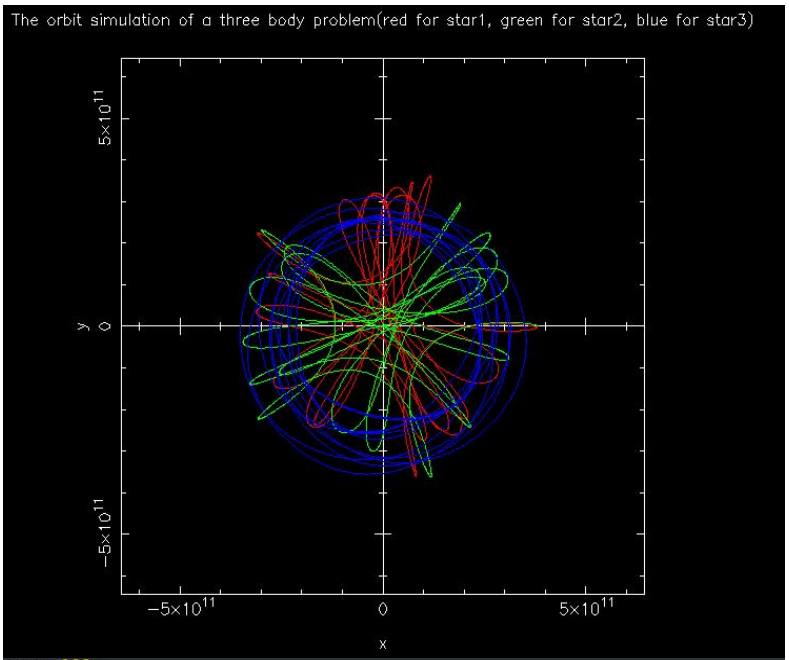
Figure 19



Name:Siheng Zhao

non-periodic:

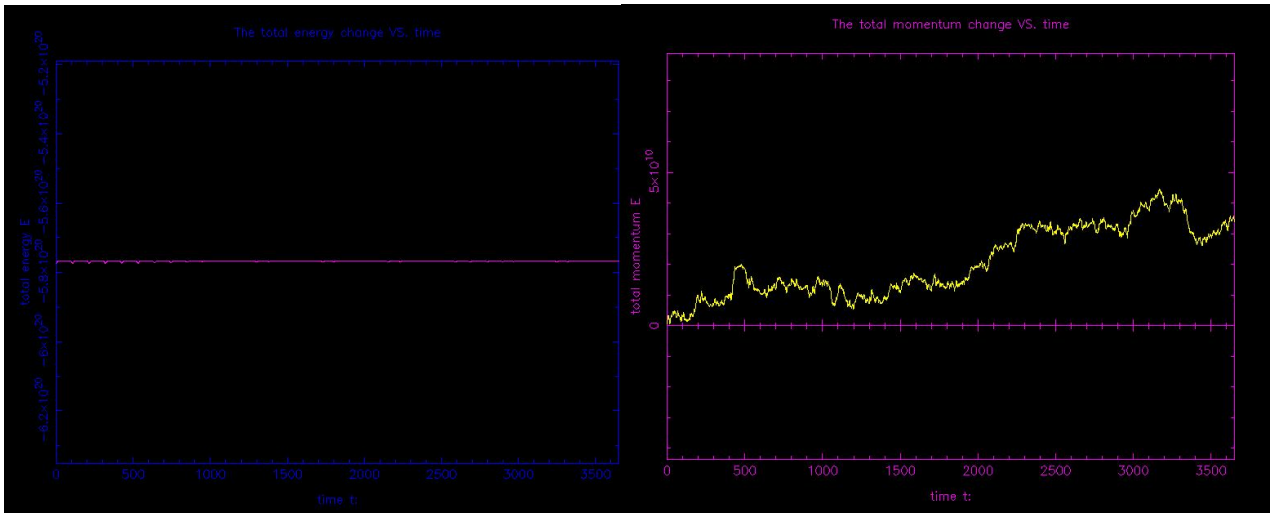
Orbit 1(Figure 20):




```
case 103:
    //(unstable model 4)
    s1 = {1*1.5e31 , 6.963e8 , -1.1889693067e11 , 0 , 0 , 0.8042120498e5};
    s2 = {1*1.5e31 , 6.963e8 , 3.8201881837e11 , 0 , 0 , 0.0212794833e5};
    s3 = {1*1.5e31 , 6.963e8 , -2.631218877e11 , 0 , 0 , -0.8254915331e5};
    scale = 1;
    break;
```

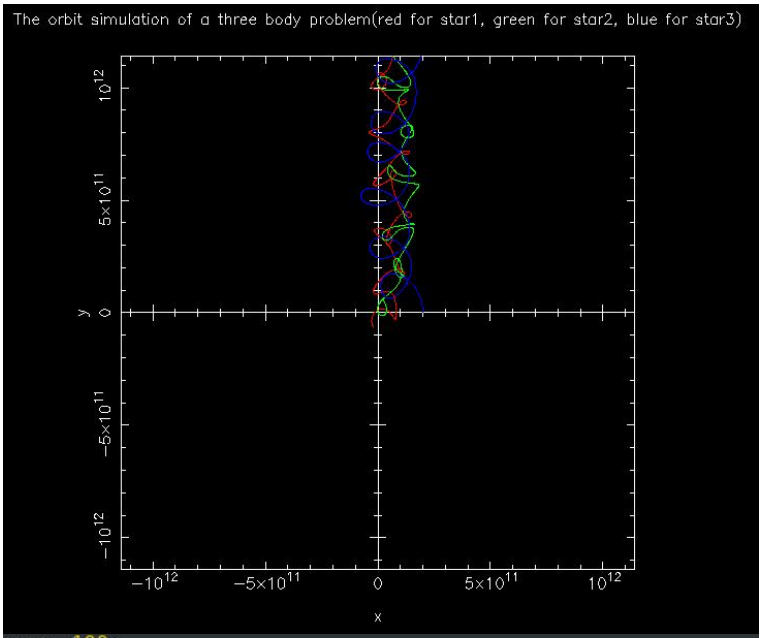
```
the error of the total energy is((e_max - e_min)/e_initial):-0.00116886
the error of the total momentum is((p_max - p_min)/p_initial):60.3008
```

Figure 20



Name:Siheng Zhao

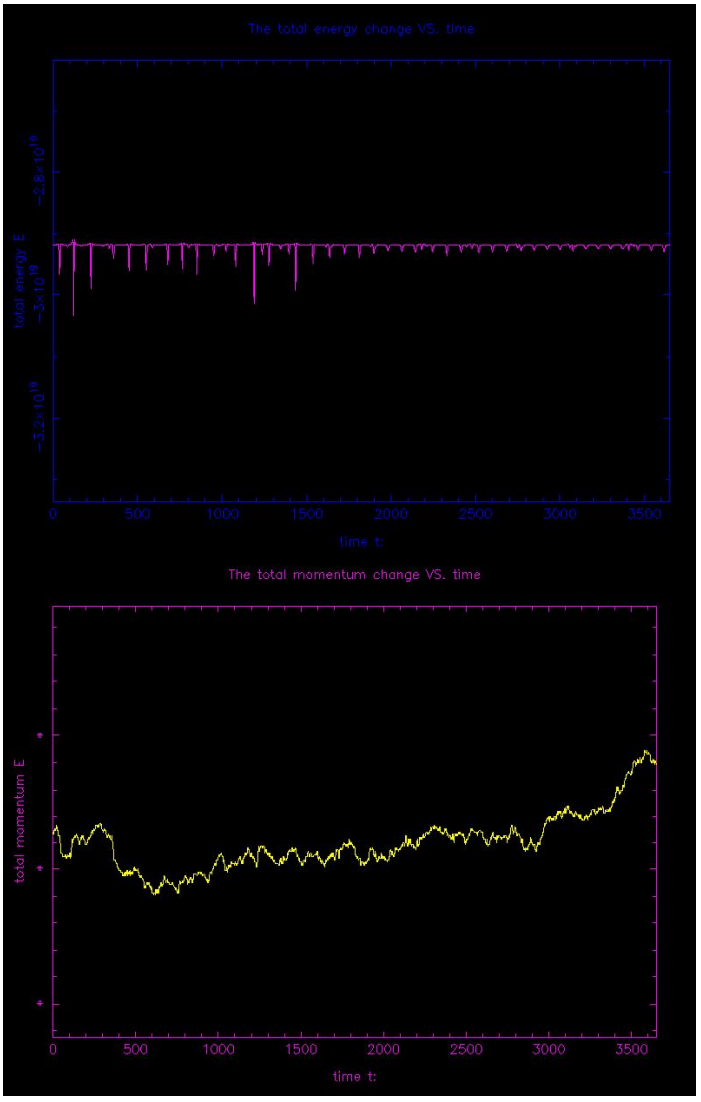
Orbit 2(Figure 21):




```
case 100:
    //(unstable model 1)
    s1 = {1.989e30 , 6.963e8 , -2e10 , -6e10 , -1e4 , 1e4};
    s2 = {1.989e30 , 6.963e8 , 2e10 , 6e10 , 1e4 , -1e4};
    s3 = {1.989e30 , 1.73e9 , 2e11 , 0, 0, 36442};
    scale = 5;
    break;
```

```
the error of the total energy is((e_max - e_min)/e_initial):-0.042218
the error of the total momentum is((p_max - p_min)/p_initial):7.40684e-06
```

Figure 21



extension code testing:

Due to the instability of the three-body problem, in addition to judging whether the code runs normally by conservation of energy and momentum like the two-body problem, it is also possible to verify whether the orbit is stable by entering some special initial state to see whether the orbit is going stable(special solution).

For example(Figure22), in this picture the orbit is almost the same with the one I simulate(Figure 18).

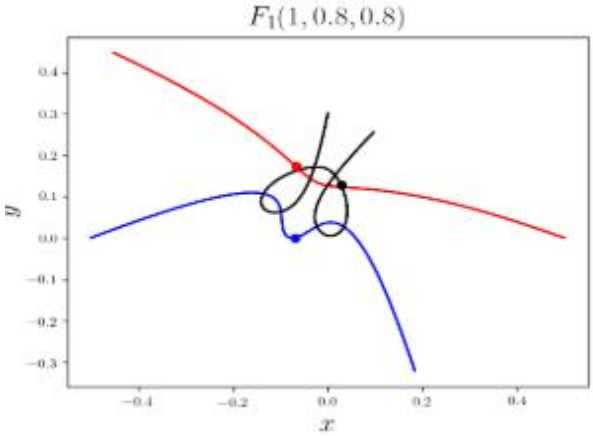


Figure 22

Not only Figure18, you can also find the orbit and initial parameters for Figure 17/19 in this three websites:
numericaltank.sjtu.edu.cn/three-body/three-body.htm & 3BODY GALLERY: SEQUENCES & Periodic Planar Three-Body Orbits / Ricky Reusser | Observable

Scientific interpretation & testing

In this case(Figure23), I want to increase earth orbit period to 10 years, according to Kepler third law, T^2 proportional to a^3. radius should increase 10^(2/3) times(Figure23red box), velocity should decrease 10^(1/3) times(Figure23blue box), according to GM/r^2 = v^2/r , and we give sun and earth a same velocity vector, so that we can see how many rotation they goes(Figure23). After 10 years, there were only 1 rotation in this simulation compare to the normal earth orbit(Figure).

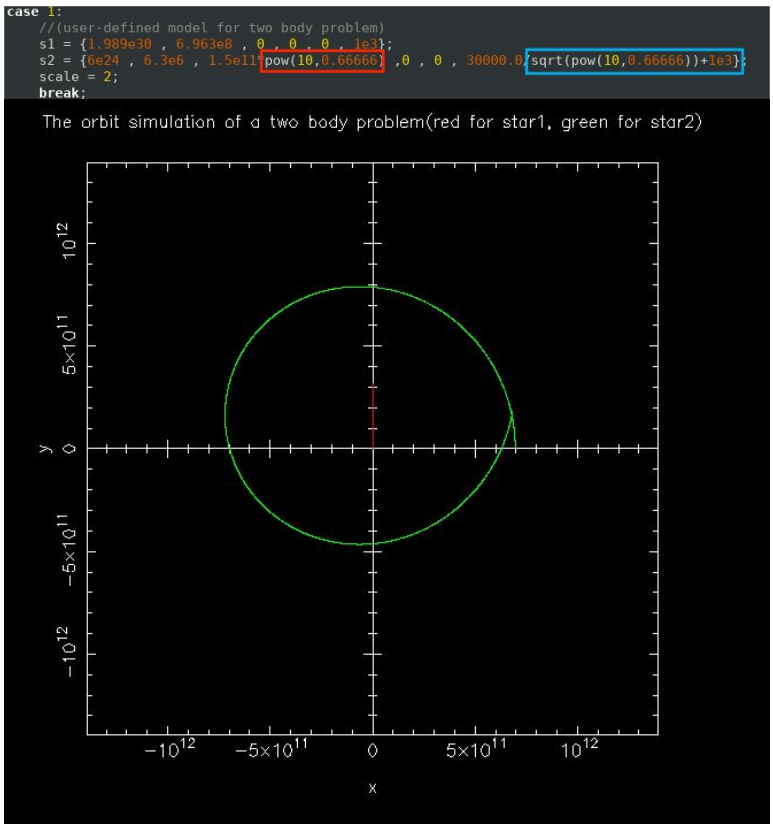
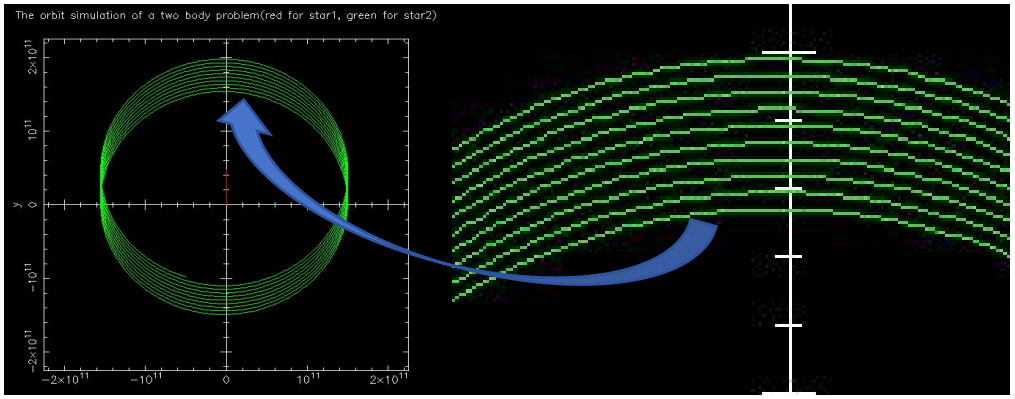


Figure 24

Figure 25



Disadvantage & improvement I should make

1. After I finished my project I found that for some large mass object or samll radius orbit, there will be only a straight line in the graph. Is because that, I set 86400s(one day) as the time step, this will only be appropriate for some acceleration(mass, orbit radius), if the acceleration is too large it will throw the star out from it gravitational potential well(V,star > escape velocity)even if it didn't have that large velocity, so I should set up a self-adaption time step. It can chose an appropriate time step according to the acceleration and there distance. For example, the orbit in figure 18 can only run well under 86400s, if I take 864000s, the orbit of the system goes very bad(Figure 24).

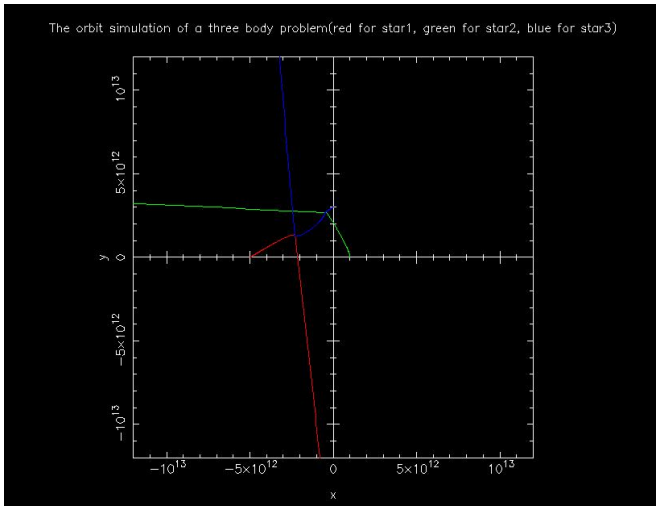


Figure 24

2. This program can only simulate two and three body problem, I should make some adjustment, so that user can define how many body in this program.

Name:Siheng Zhao

Code:

```
#include<iostream>

#include<cmath>

#include"cpgplot.h"

#include <algorithm>

#include<cmath>


//the gravitational constant G

double G = 6.67430e-11;


//set the update time interval

double dt = 86400; //86400s = 1 day


//set how long do you want to simulate

int t = 3650; //in days


//set the parameter for what time is now

int day = 0;


//set how many bodies do you want to simulate(2 or 3)

int body_num;




//the maximum distance between two star

float dmax;




//set the parameter for the star
```

Name:Siheng Zhao

```
struct star{
        double m;
        double r;
        float x , y;
        float vx , vy;
};


star s1, s2, s3;




//distance between two star
double d(const star &a, const star &b){
        double distance;
        distance = sqrt(pow(a.x-b.x,2)+pow(a.y-b.y,2));
        return distance;
}




//speed of a star
double v(const star &a){
        return sqrt(pow(a.vx,2)+pow(a.vy,2));
}




//seting the coefficient for the graph(limit of x&y on the graph = the largest distance(dmax)*scale)
//defult scale = 1.5
float scale = 1.5;
```

```
//initialiez the star parameter
//                    {mass, radius, x, y, vx, vy}


//define the model number that you want to simulate
int model;


//select the model according to you options
void model_selection(){
        //body number selection
        std::cout << "2 for two body problem simulation" << '\n' << "3 for three body problem simulation" << '\n' << '\n';
        std::cin >> body_num;


        //model selection prompt
        if (body_num == 2){
        std::cout <<"two body problem model selection:" << '\n' << '\n'<<"1 for user-defined model" << '\n' << "2 for
earth-sun system model"<< '\n' << "3 for earth-sun system model with a non-zero initial momentum" << '\n' <<"4 for
binary star system" << '\n' << "5 for binary star system with a non-zero momentum"    <<    '\n' << '\n';
        }
        else if(body_num == 3){
        std::cout <<"three body problem model selection:" << '\n' << '\n'<<"1 for user-defined model" << '\n' <<"2 for the
third Lagrangian Points with a star at rest" << '\n' << "3 for the third Lagrangian Points with a moving star" << '\n' <<
```

```
"4 for earth-sun-mars system" << '\n' << "5 for resonance system(simulate for 300 earth-year, about 1-2min in real
world)" << '\n' <<"6 for a star passing a binary system" << '\n' << "7 for a planet orbiting a binary system" << '\n' <<'\n'
<< "special case solution for three body problems:" << '\n'<< "8 for stable three body problem's special case solution
code name-'Figure 8'" << '\n' << "9 for stable three body problem's special case solution code name-'Free Fall F1'"<< '\n'
<< "10 for stable three body problem's special case solution code name-'Double Ovals'" << '\n'<< '\n' << "unstable
system:"<< '\n'<<"100 for unstable system 1" <<'\n'<<"101 for unstable system 2" << '\n' <<"102 for unstable system 3"
<< '\n'<<"103 for unstable system 4" << '\n'    << '\n';

        }




        //select the model you want

        std::cin >> model;




        //model selecting


        //two bodymodel

        if (body_num == 2){

                switch(model){

                        case 1:

                                //(user-defined model for two body problem)

                                s1 = {1.989e30 , 6.963e8 , 0 , 0 , 0 , 1e3};
                                        //<-- user-defined model inputer here

                                s2 = {6e24 , 6.3e6 , 1.5e11*pow(10,0.66666) ,0 , 0 , 30000.0/sqrt(pow(10,0.66666))+1e3};
                //<-- user-defined model inputer here

                                scale = 2;
                                                        //<-- user-defined scale inputer here

                                break;



                        case 2:

                                //(earth-sun system)

                                s1 = {1.989e30 , 6.963e8 , 0 , 0 , 0 , 0};                              //<-- Warning! DO NOT change
```

```
this line!!

                    s2 = {6e24 , 6.3e6 , 1.5e11 ,0 , 0 , 30000.0};                    //<-- Warning! DO NOT change
this line!!

                    scale = 1.5;                                                                    //<-- Warning!
DO NOT change this line!!

                    break;


                case 3:

                    //(earth-sun system with a non-zero initial momentum)

                    s1 = {1.989e30 , 6.963e8 , 0 , 0 , 0 , 1.5e2};                    //<-- Warning! DO NOT
change this line!!

                    s2 = {6e24 , 6.3e6 , 1.5e11 ,0 , 0 , 30000.0+1.5e2};              //<-- Warning! DO NOT
change this line!!

                    scale = 1.5;                                                                    //<-- Warning!
DO NOT change this line!!

                    break;


                case 4:

                    //(binary star system)

                    s1 = {1.989e30 , 6.963e8 , -2e10 , -6e10 , -1e4 , 1e4};           //<-- Warning! DO NOT change
this line!!

                    s2 = {1.989e30 , 6.963e8 , 2e10 , 6e10 , 1e4 , -1e4};      //<-- Warning! DO NOT change this
line!!

                    scale = 1.5;                                                                    //<--
Warning! DO NOT change this line!!

                    break;


                case 5:

                    //(binary star system with a non-zero momentum)

                    s1 = {1.989e30 , 6.963e8 , -2e10 , -6e10 , -1.5e4 , 1e4};  //<-- Warning! DO NOT change this
line!!

                    s2 = {4e30 , 6.963e8 , 2e10 , 6e10 , 1e4 , -1e4};                 //<-- Warning! DO NOT change
this line!!
```

```
                scale = 20;                                                    //<--
Warning! DO NOT change this line!!
                break;
            }
        }

        //three body model
        else if (body_num == 3){
            switch(model){
                case 1:
                    //(user-defined model for three body problem)
                    s1 = {1*1.5e32 , 6.963 , -0.5e13 , 0 , 0 , 0};          //<-- user-defined model inputer here
                    s2 = {1*1.5e30 , 6.963e8 , 1e12 , 0 , 0.3471168881e4 , 0.5327249454e4};
        //<-- user-defined model inputer here
                    s3 = {0.8*1.5e32 , 6.963 , 0.0009114239e13 , 0.3019805958e13 , 0 , 0};    //<-- user-defined
model inputer here
                    scale = 2;
                    //<-- user-defined scale inputer here
                    dt = 10 *dt;
                    //t=1e4;
                    //G=1;
                    break;

                case 2:
                    //(the third Lagrangian Points with a star at rest)
                    s1 = {1.989e30 , 6.963e8 , 0 , 0 , 0 , 0};              //<-- Warning! DO NOT change
this line!!
                    s2 = {6e24 , 6.3e6 , 1.5e11 ,0 , 0 , 30000.0};          //<-- Warning! DO NOT change
this line!!
                    s3 = {7.34e22 , 1.73e6 , -1.5e11 , 0, 0, -30000.0};     //<-- Warning! DO NOT change
this line!!
                    scale = 1;                                                 //<--
```

```
Warning! DO NOT change this line!!
                    break;


                case 3:
                    //(the third Lagrangian Points with a moving star)
                    s1 = {1.989e30 , 6.963e8 , 0 , 0 , 1000 , 1000};          //<-- Warning! DO NOT change
this line!!
                    s2 = {6e24 , 6.3e6 , 1.5e11 ,0 , 0 , 30000.0};            //<-- Warning! DO NOT change
this line!!
                    s3 = {7.34e22 , 1.73e6 , -1.5e11 , 0, 0, -30000.0};       //<-- Warning! DO NOT change
this line!!
                    scale = 2;                                                //<--
Warning! DO NOT change this line!!
                    break;


                case 4:
                    //(earth-sun-mars)
                    s1 = {1.989e30 , 6.963e8 , 0 , 0 , 0 , 0};                //<-- Warning! DO NOT change
this line!!
                    s2 = {6e24 , 6.3e6 , 1.5e11 ,0 , 0 , 30000.0};            //<-- Warning! DO NOT change
this line!!
                    s3 = {6.4171e24 , 3.3895e6 , 2.2794e11 , 0 , 0 , 24000.0};  //<-- Warning! DO NOT change this
line!!
                    scale = 1.5;                                              //<-- Warning!
DO NOT change this line!!
                    break;


                case 5:
                    //(resonance system)
                    s1 = {1.989e30 , 6.963e8 , 0 , 0 , 0 , 0};                //<-- Warning! DO NOT change
this line!!
                    s2 = {6e24 , 6.3e6 , 1.5e11 ,0 , 0 , 30000.0};            //<-- Warning! DO NOT change
this line!!
```

```
                    s3 = {6e26 , 1.73e6 , 2.38e11 , 0, 0, 23674.0};                    //<-- Warning! DO NOT change
this line!!

                    scale = 1.5;                                                                    //<-- Warning!
DO NOT change this line!!

                    t = 1.095e5;

                    break;


                case 6:

                    //(a star passing a binary system)

                    s1 = {1.989e30 , 6.963e8 , -2e10 , -6e10 , -1e4 , 1e4};            //<-- Warning! DO NOT change
this line!!

                    s2 = {1.989e30 , 6.963e8 , 2e10 , 6e10 , 1e4 , -1e4};        //<-- Warning! DO NOT change this
line!!

                    s3 = {7.34e22 , 1.73e6 , -1.5e11 , 0, 0, -30000.0};                //<-- Warning! DO NOT change
this line!!

                    scale = 1;                                                                      //<--
Warning! DO NOT change this line!!

                    break;


                case 7:

                    //(a planet orbiting a binary system)

                    s1 = {1.989e30 , 6.963e8 , -2e10 , -6e10 , -1e4 , 1e4};            //<-- Warning! DO NOT change
this line!!

                    s2 = {1.989e30 , 6.963e8 , 2e10 , 6e10 , 1e4 , -1e4};        //<-- Warning! DO NOT change this
line!!

                    s3 = {1.989e24 , 1.73e9 , 2e11 , 0, 0, 36442};                    //<-- Warning! DO NOT change
this line!!

                    scale = 1;                                                                      //<--
Warning! DO NOT change this line!!

                    break;


                //stable three body problem's special case solution
```

```
                case 8:

                    //(stable orbit-'Figure "8"')

                    s1 = {1*1.5e30 , 6.963e8 , -1e12 , 0 , 0.3471168881e4 , 0.5327249454e4};      //<-- Warning! DO
NOT change this line!!

                    s2 = {1*1.5e30 , 6.963e8 , 1e12 , 0 , 0.3471168881e4 , 0.5327249454e4};       //<-- Warning!
DO NOT change this line!!

                    s3 = {1*1.5e30 , 6.963e8 , 0 , 0 , -0.3471168881*2e4 , -0.5327249454*2e4};   //<-- Warning! DO
NOT change this line!!

                    scale = 1;
                //<-- Warning! DO NOT change this line!!

                    break;



                case 9:

                    //(stable orbit-'Free Fall F1')

                    s1 = {1*1.5e32 , 6.963 , -0.5e13 , 0 , 0 , 0};                                  //<-- Warning!
DO NOT change this line!!

                    s2 = {0.8*1.5e32 , 6.963 , 0.5e13 , 0 , 0 , 0};                                 //<-- Warning!
DO NOT change this line!!

                    s3 = {0.8*1.5e32 , 6.963 , 0.0009114239e13 , 0.3019805958e13 , 0 , 0};//<-- Warning! DO NOT
change this line!!

                    scale = 1;
            //<-- Warning! DO NOT change this line!!

                    break;



                case 10:

                    //(stable orbit-'Double Ovals')

                    s1 = {1*1.5e21*1.3, 0 , 0.486657678894505e9*1.3,0.755041888583519e9*1.3,
-0.182709864466916e1,0.363013287999004e1};          //<-- Warning! DO NOT change this line!!

                    s2 = {1*1.5e21*1.3, 0 , -0.681737994414464e9*1.3,0.29366023319721e9*1.3,
-0.579074922540872e1,-0.748157481446087e1};    //<-- Warning! DO NOT change this line!!

                    s3 = {1*1.5e21*1.3, 0 , -0.02259632746864e9*1.3,-0.612645601255358e9*1.3,
0.761784787007641e1,0.385144193447218e1};          //<-- Warning! DO NOT change this line!!

                    scale = 2;
                                                                    //<-- Warning! DO NOT change
```

```
this line!!
                    break;

                //unstable three body problem's orbit

                case 100:
                    //(unstable model 1)
                    s1 = {1.989e30 , 6.963e8 , -2e10 , -6e10 , -1e4 , 1e4};          //<-- Warning! DO NOT change
this line!!
                    s2 = {1.989e30 , 6.963e8 , 2e10 , 6e10 , 1e4 , -1e4};        //<-- Warning! DO NOT change this
line!!
                    s3 = {1.989e30 , 1.73e9 , 2e11 , 0, 0, 36442};                //<-- Warning! DO NOT change
this line!!
                    scale = 5;                                                                    //<--
Warning! DO NOT change this line!!
                    break;

                case 101:
                    //(unstable model 2)
                    s1 = {1.989e30 , 6.963e8 , -3e11 , -2e11 , -1.5e4 , 1e4};    //<-- Warning! DO NOT change this
line!!
                    s2 = {1.989e30 , 6.963e8 , 3e11 , -2e11 , 1e4 , -1e4};          //<-- Warning! DO NOT change
this line!!
                    s3 = {1.989e30 , 6.963e8 , 0 , 3e11 , -1e4 , 0};                //<-- Warning! DO NOT change
this line!!
                    scale = 5;                                                                    //<--
Warning! DO NOT change this line!!
                    break;

                case 102:
                    //(unstable model 3)
                    s1 = {1.989e30 , 6.963e8 , -3e11 , -2e11 , 3e4 , 2e4};          //<-- Warning! DO NOT change
this line!!
```

```
                    s2 = {1.989e30 , 6.963e8 , 3e11 , -2e11 , -2e4 , 0};          //<-- Warning! DO NOT change this
line!!
                    s3 = {1.989e30 , 6.963e8 , 0 , 3e11 , -1e4 , -1e4};                //<-- Warning! DO NOT change
this line!!
                    scale = 5;                                                                                    //<--
Warning! DO NOT change this line!!
                    break;


                case 103:
                    //(unstable model 4)
                    s1 = {1*1.5e31 , 6.963e8 , -1.1889693067e11 , 0 , 0 , 0.8042120498e5};        //<-- user-defined
model inputer here
                    s2 = {1*1.5e31 , 6.963e8 , 3.8201881837e11 , 0 , 0 , 0.0212794833e5}; //<-- user-defined model
inputer here
                    s3 = {1*1.5e31 , 6.963e8 , -2.631218877e11 , 0 , 0 , -0.8254915331e5}; //<-- user-defined model
inputer here
                    scale = 1;
            //<-- user-defined scale inputer here
                    break;


            }


        }


}
```

```
//gravitational force & accelearation component on each direction between two/three stars
```

Name:Siheng Zhao

```
// two bodies:
        double ax2(const star &a, const star &b){
                return G*b.m/pow(d(a, b),2)*(b.x-a.x)/d(a, b);
        }

        double ay2(const star &a, const star &b){
                return G*b.m/pow(d(a, b),2)*(b.y-a.y)/d(a, b);
        }

//three bodies:
        double ax3(const star &a, const star &b, const star &c){
                return G*b.m/pow(d(a, b),2)*(b.x-a.x)/d(a, b)+G*c.m/pow(d(a, c),2)*(c.x-a.x)/d(a, c);

        }

        double ay3(const star &a, const star &b, const star &c){
                return G*b.m/pow(d(a, b),2)*(b.y-a.y)/d(a, b)+G*c.m/pow(d(a, c),2)*(c.y-a.y)/d(a, c);
        }

//gravitational potential energy
double eu(const star &a, const star &b){
        return -G*a.m*b.m/d(a, b);
}
```

Name:Siheng Zhao

```
//kinetic energy
double ek(const star &a ){
	return 0.5*a.m*pow(v(a),2);
}



//total energy for two/three bodies
//for two bodies
double e2(const star &a, const star &b){
	return eu(a, b) + ek(a) + ek(b);
}
//for three bodies
double e3(const star &a, const star &b, const star &c){
	return eu(a, b) + eu(a, c) +eu(b, c) + ek(a) + ek(b) +ek(c);
}



//momentum calculation
//total momentum for two body
double p2(const star &a,const star &b){
	return sqrt(pow(a.m*a.vx+b.m*b.vx,2)+pow(a.m*a.vy+b.m*b.vy,2));
}

//total momentum for three body
double p3(const star &a, const star &b, const star &c){
	return sqrt(pow(a.m*a.vx+b.m*b.vx+c.m*c.vx,2)+pow(a.m*a.vy+b.m*b.vy+c.m*c.vy,2));
}
```

```
//report status for two/three stars
//for two bodies
void report2(const star &a, const star &b){
	//report the time
	std::cout << "in day" << day << '\n';

	//report star 1
	std::cout << "the position of star1 is: " << a.x << "," << a.y <<'\n'<<"the speed of star1 is: " << v(a) << '\n';
	std::cout << "v1x = " << a.vx << " v1y = " << a.vy << '\n';
	std::cout << "a1x = " << ax2(a,b)<< " a1y = " << ay2(a,b) << '\n';
	std::cout << "the kinetic energy of star1 is: " << ek(a) << '\n' << '\n';

	//report star 2
	std::cout << "the position of star2 is: " << b.x << "," << b.y <<'\n'<<"the speed of star2 is: " << v(b) << '\n';
	std::cout << "v2x = " << b.vx << " v2y = " << b.vy << '\n';
	std::cout << "a2x = " << ax2(b,a)<< " a2y = " << ay2(b,a) << '\n';
	std::cout << "the kinetic energy of star2 is: " << ek(b) << '\n' << '\n';

	//report the potential energy between the two star
	std::cout << "the potential energy between star1 and star2 is: " << eu(a,b) << '\n';

	//report the total energy in the system
	std::cout << "the total energy in this system is: " << e2(a,b) << '\n' << '\n' ;

	//report the total momentun of the system
	std::cout << "the total momentum for the system is: " << p2(a,b) << '\n' << '\n' << '\n' ;
}
```

Name:Siheng Zhao

```
//for three bodies
void report3(const star &a, const star &b, const star &c){
        //report the time
        std::cout << "in day" << day << '\n';

        //report star 1
        std::cout << "the position of star1 is: " << a.x << "," << a.y <<'\n'<<"the speed of star1 is: " << v(a) << '\n';
        std::cout << "v1x = " << a.vx << " v1y = " << a.vy << '\n';
        std::cout << "a1x = " << ax3(a,b,c)<< " a1y = " << ay3(a,b,c) << '\n';
        std::cout << "the kinetic energy of star1 is: " << ek(a) << '\n' << '\n';

        //report star 2
        std::cout << "the position of star2 is: " << b.x << "," << b.y <<'\n'<<"the speed of star2 is: " << v(b) << '\n';
        std::cout << "v2x = " << b.vx << " v2y = " << b.vy << '\n';
        std::cout << "a2x = " << ax3(b,a,c)<< " a2y = " << ay3(b,a,c) << '\n';
        std::cout << "the kinetic energy of star2 is: " << ek(b) << '\n' << '\n';

        //report star 3
        std::cout << "the position of star3 is: " << c.x << "," << c.y <<'\n'<<"the speed of star2 is: " << v(c) << '\n';
        std::cout << "v3x = " << c.vx << " v3y = " << c.vy << '\n';
        std::cout << "a3x = " << ax3(c,b,a)<< " a3y = " << ay3(c,b,a) << '\n';
        std::cout << "the kinetic energy of star3 is: " << ek(c) << '\n' << '\n';

        //report the potential energy between star a & b
        std::cout << "the potential energy between star1 and star2 is: " << eu(a,b) << '\n';

        //report the potential energy between star a & c
        std::cout << "the potential energy between star1 and star3 is: " << eu(a,c) << '\n';
```

```
        //report the potential energy between star b & c
        std::cout << "the potential energy between star2 and star3 is: " << eu(b,c) << '\n';

        //report the total energy in the system
        std::cout << "the total energy in this system is: " << e3(a,b,c) << '\n' << '\n';

        //report the total momentun of the system
        std::cout << "the total momentum for the system is: " << p3(a,b,c) << '\n' << '\n' << '\n' ;
}

int main(){
        //initialize the star parameter according to the model
        model_selection();

        //half step of t
        float hv1x , hv2x , hv3x , hv1y , hv2y , hv3y;

        //define speed of the 3 star
        float v1x[t+1] , v1y[t+1];
        float v2x[t+1] , v2y[t+1];
        float v3x[t+1] , v3y[t+1];

        //define the position of the 3 star
        float x1[t+1] , y1[t+1];
```

```
float x2[t+1] , y2[t+1];

float x3[t+1] , y3[t+1];


float time[t+1];


//define the total energy/momentum arrays for calculating the

float e[t+1],p[t+1];


//define the maximun energy/momentum and the minimum energy;

float emax, emin, pmax, pmin;




//store the initial v in to the arrays:

//vx:

v1x[0] = s1.vx;

v2x[0] = s2.vx;

v3x[0] = s3.vx;

//vy

v1y[0] = s1.vy;

v2y[0] = s2.vy;

v3y[0] = s3.vy;


//store the initial position in to the arrays:

//x:

x1[0] = s1.x;

x2[0] = s2.x;

x3[0] = s3.x;

//y:

y1[0] = s1.y;
```

```
        y2[0] = s2.y;
        y3[0] = s3.y;

        //store the initial time into the arrays
        time[0] = 0;

        std::cout <<'\n'<< body_num <<" body problem simulation start." << '\n' << '\n';

//report the initial state for the two/three star
        //for two bodies:
        if(body_num == 2){
        report2(s1,s2);

        //record the initial energy/momentum
        e[0] = e2(s1,s2)*1e-20;
        p[0] = p2(s1,s2)*1e-20;
        }

        //for three bodies
        else{
        report3(s1,s2,s3);

        //record the initial energy/momentum
        e[0] = e3(s1,s2,s3)*1e-20;
        p[0] = p3(s1,s2,s3)*1e-20;
        }
```

```
        //open pgplot
        if (!cpgopen("/XWINDOW")) return 1;

        //plot line setting
        cpgslw(1.0);



//leapfrog calculation:
        //leapfrog calculation for two bodies
        if (body_num == 2){


                dmax = d(s1,s2);

                //two body problem's leap frog calculation:
                for(int i =1;i<=t;i++){

                        //update & store the time of the system
                        day = i;
                        time[i] = day;



                        //update the first half step for star 1
                        hv1x = v1x[i-1] + 0.5*dt*ax2(s1,s2);
                        hv1y = v1y[i-1] + 0.5*dt*ay2(s1,s2);
                        //update the half step for star 2
                        hv2x = v2x[i-1] + 0.5*dt*ax2(s2,s1);
```

```
hv2y = v2y[i-1] + 0.5*dt*ay2(s2,s1);



//update the position for star 1

x1[i] = x1[i-1] + hv1x*dt;

s1.x = x1[i];

y1[i] = y1[i-1] + hv1y*dt;

s1.y = y1[i];

//update the position for star 2

x2[i] = x2[i-1] + hv2x*dt;

s2.x = x2[i];

y2[i] = y2[i-1] + hv2y*dt;

s2.y = y2[i];


//check whether the two stars collided on each other

if (d(s1,s2)<=s1.r + s2.r){

std::cout << "star1 and star2 collided in day " << day << '\n';

break;

}




//update the final half step for star 1

v1x[i] = hv1x + 0.5*dt*ax2(s1,s2);

s1.vx = v1x[i];

v1y[i] = hv1y + 0.5*dt*ay2(s1,s2);

s1.vy = v1y[i];


//update the final half step for star 2

v2x[i] = hv2x + 0.5*dt*ax2(s2,s1);
```

```
            s2.vx = v2x[i];
            v2y[i] = hv2y + 0.5*dt*ay2(s2,s1);
            s2.vy = v2y[i];




            //store the total energy/momentum
            e[i] = e2(s1,s2)*1e-20;
            p[i] = p2(s1,s2)*1e-20;




            //report the status of the two stars
            report2(s1,s2);
        }



    //calculate emax and emin
    emax = *std::max_element(e, e + t+1);
    emin = *std::min_element(e, e + t+1);

    //calculate pmax and pmin
    pmax = *std::max_element(p, p + t+1);
    pmin = *std::min_element(p, p + t+1);



    //report the error of the total energy for the simulation
    std::cout<< "the error of the total energy is((e_max - e_min)/e_initial):" << (emax - emin)/e[0] << '\n';
```

```
//report the error of the total momentum for the simulation
if(p[0] == 0){
        std::cout<< "the error of the total momentum is((p_max - p_min)/p_initial):" << 0 << '\n';
}
else{
        std::cout<< "the error of the total momentum is((p_max - p_min)/p_initial):" << (pmax - pmin)/p[0] << '\n';
}
```

```
//plot the orbit, energy and momentum
//orbit:
//plot axes setting (maximum of x&y axes = the largest distance between those stars)
cpgenv(-scale*dmax,scale*dmax,-scale*dmax,scale*dmax,1,1);
//label axes
cpglab("x","y","The orbit simulation of a two body problem(red for star1, green for star2)");
//plot the graph of star1
cpgsci(2);
cpgline(t+1,x1,y1);
//plot the graph of star2
cpgsci(3);
cpgline(t+1,x2,y2);
```

```
//energy & momentum:
//plot the graph for energy & momentum
```

```
//energy:
cpgenv(time[0],time[t],emin-0.1*fabs(emin),emax+0.1*fabs(emax),0,1);
cpglab("time t:","total energy E","The total energy change VS. time");
cpgsci (6);
cpgline(t+1,time,e);

//momentum:
if(pmax == pmin){
        if(pmax == 0 && pmin == 0){
        cpgenv(time[0],time[t],-1,1,0,1);

        }else{
        cpgenv(time[0],time[t],-1.01*fabs(pmin),1.01*fabs(pmax),0,1);
        }

}else{
        cpgenv(time[0],time[t],pmin-pmax+pmin,pmax+pmax-pmin,0,1);
}

cpglab("time t:","total momentum E","The total momentum change VS. time");
cpgsci (7);
cpgline(t+1,time,p);

cpgclos();

}
```

```
//leapfroge calculation for three bodies
else{

    dmax = std::max(d(s1,s2),std::max(d(s1,s3),d(s2,s3)));

    //three body problem's leapfrog claculation
    for(int i =1;i<=t;i++){

        //update the time of the system
        day = i;
        time[i] = day;

        //update the first half step for star 1
        hv1x = v1x[i-1] + 0.5*dt*ax3(s1,s2,s3);
        hv1y = v1y[i-1] + 0.5*dt*ay3(s1,s2,s3);
        //update the first half step for star 2
        hv2x = v2x[i-1] + 0.5*dt*ax3(s2,s1,s3);
        hv2y = v2y[i-1] + 0.5*dt*ay3(s2,s1,s3);
        //update the first half step for star 3
        hv3x = v3x[i-1] + 0.5*dt*ax3(s3,s1,s2);
        hv3y = v3y[i-1] + 0.5*dt*ay3(s3,s1,s2);

        //update the position for star 1
        x1[i] = x1[i-1] + hv1x*dt;
        s1.x = x1[i];
        y1[i] = y1[i-1] + hv1y*dt;
        s1.y = y1[i];
```

```
//update the position for star 2
x2[i] = x2[i-1] + hv2x*dt;
s2.x = x2[i];
y2[i] = y2[i-1] + hv2y*dt;
s2.y = y2[i];
//update the position for star 3
x3[i] = x3[i-1] + hv3x*dt;
s3.x = x3[i];
y3[i] = y3[i-1] + hv3y*dt;
s3.y = y3[i];

//check whether any of the the two stars collided on each other
if (d(s1,s2)<=s1.r + s2.r){
std::cout << "star1 and star2 collided in day " << day << '\n';
break;
}
else if(d(s1,s3)<=s1.r + s3.r){
std::cout << "star1 and star3 collided in day " << day << '\n';
break;
}
else if(d(s3,s2)<=s3.r + s2.r){
std::cout << "star3 and star2 collided in day " << day << '\n';
break;
}

//update the final half step for star 1
v1x[i] = hv1x + 0.5*dt*ax3(s1,s2,s3);
s1.vx = v1x[i];
v1y[i] = hv1y + 0.5*dt*ay3(s1,s2,s3);
```

```
            s1.vy = v1y[i];


            //update the final half step for star 2
            v2x[i] = hv2x + 0.5*dt*ax3(s2,s1,s3);
            s2.vx = v2x[i];
            v2y[i] = hv2y + 0.5*dt*ay3(s2,s1,s3);
            s2.vy = v2y[i];


            //update the final half step for star 3
            v3x[i] = hv3x + 0.5*dt*ax3(s3,s1,s2);
            s3.vx = v3x[i];
            v3y[i] = hv3y + 0.5*dt*ay3(s3,s1,s2);
            s3.vy = v3y[i];




            //store the total energy/momentum
            e[i] = e3(s1,s2,s3)*1e-20;
            p[i] = p3(s1,s2,s3)*1e-20;



            //report the status of the three stars
            report3(s1,s2,s3);
        }


    //calculate emax and emin
    emax = *std::max_element(e, e + t+1);
    emin = *std::min_element(e, e + t+1);
```

```
//calculate pmax and pmin
pmax = *std::max_element(p, p + t+1);
pmin = *std::min_element(p, p + t+1);



//report the error of the total energy for the simulation
std::cout<< "the error of the total energy is((e_max - e_min)/e_initial):" << (emax - emin)/e[0] << '\n';


//report the error of the total momentum for the simulation


if(p[0] == 0){
        std::cout<< "the error of the total momentum is((p_max - p_min)/p_initial):" << 0 << '\n';
}
else{
        std::cout<< "the error of the total momentum is((p_max - p_min)/p_initial):" << (pmax - pmin)/p[0] << '\n';
}









//plot axes setting (maximum of x&y axes = the largest distance between those stars)
cpgenv(-scale*dmax,scale*dmax,-scale*dmax,scale*dmax,1,1);
//label axes
cpglab("x","y","The orbit simulation of a three body problem(red for star1, green for star2, blue for star3)");
//plot the graph of star1
cpgsci(2);
cpgline(t+1,x1,y1);
//plot the graph of star2
```

```
cpgsci(3);
cpgline(t+1,x2,y2);
//plot the graph of star3
cpgsci(4);
cpgline(t+1,x3,y3);




//energy & momentum:
//plot the graph for energy & momentum
//energy:
cpgenv(time[0],time[t],emin-0.1*fabs(emin),emax+0.1*fabs(emax),0,1);
cpglab("time t:","total energy E","The total energy change VS. time");
cpgsci (6);
cpgline(t+1,time,e);


//momentum:
if(pmax == pmin){
        if(pmax == 0 && pmin == 0){
        cpgenv(time[0],time[t],-1,1,0,1);


        }else{
        cpgenv(time[0],time[t],-1.01*fabs(pmin),1.01*fabs(pmax),0,1);
        }




}else{
        cpgenv(time[0],time[t],pmin-pmax+pmin,pmax+pmax-pmin,0,1);
```

```
        }


        cpglab("time t:","total momentum E","The total momentum change VS. time");
        cpgsci (7);
        cpgline(t+1,time,p);


        cpgclos();


        }




}
```